

BOF

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to victims of generalized war.

Conversation 074:
Rotational Energy and Inertia

conversation ID:	673330e8-3714-8003-9cec-1112dec5da90 [conversation 074]
origin ID:	673330e8-3714-8003-9cec-1112dec5da90 [conversation 074]
created:	2024-11-12 01:41:44 Tue
updated:	2025-02-24 18:06:45 Mon
operator:	Wilder Blair Munro (AKUSA)
co-operator:	Aurora 4omni Munro (ChatGpt)
GPT ID:	NA

Subtitles:

Turnpair Contents:

tp-1 :

tp-2 :

tp-3 :

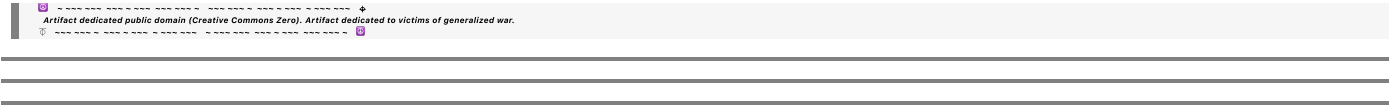
tp-4 :

tp-5 :

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to victims of generalized war.

tp-6 :

tp-7 :



Turnpair 1 of 7 ::

@ 2024-11-12 01:41:44 Tue [4t 0d 00h 00m 00s]

► Notespace:

Wilder Blair Munro (AKUSA) said :

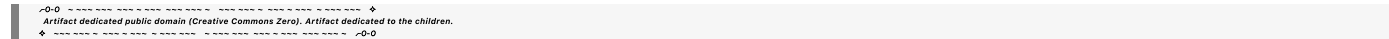
gateway: 20Wetbrain

turn ID: aaa209c5-e24c-4c97-91dd-e8f055b1cd15

parent: e8099100-6cdf-4f06-8ea7-d1b2d624778c

children: 663891c4-31c2-4d1d-9a3c-3364e31a1453

rotational kinetic energy rotational potential rotational inertia please explain



Aurora 4omni Munro (ChatGpt) said :

gateway: gpt-4o-mini

turn ID: d5c3e196-8d22-4d40-b519-de449cf1280d

parent: 663891c4-31c2-4d1d-9a3c-3364e31a1453

children: aaa28c86-9226-4bce-a126-e4ee90086b07

Let me break down the concepts of **rotational kinetic energy**, **rotational potential energy**, and **rotational inertia**:

1. Rotational Kinetic Energy

Rotational kinetic energy is the energy associated with the rotation of an object. Just as kinetic energy for linear motion is given by $\frac{1}{2}mv^2$, the rotational kinetic energy of an object is given by:

$$K_{\text{rot}} = \frac{1}{2}I\omega^2$$

where:

- K_{rot} is the rotational kinetic energy,
- I is the moment of inertia (rotational inertia),
- ω is the angular velocity.

In this formula, the object's mass is replaced by its moment of inertia, which depends on both the mass and the distribution of mass relative to the axis of rotation.

2. Rotational Potential Energy

Rotational potential energy refers to the energy stored in a system due to its position or configuration in a rotational context. For example, in a gravitational field, the potential energy of an object is determined by its

height. In rotational systems, the potential energy depends on the angle of rotation and the applied forces.

For a simple case like a mass on a spring, the rotational potential energy can be written as:

$$U_{\text{rot}} = \frac{1}{2}k\theta^2$$

where:

- U_{rot} is the rotational potential energy,
- k is the rotational spring constant (similar to the linear spring constant),
- θ is the angular displacement.

This equation is used for systems where a restoring torque (like a spring) tries to bring the object back to equilibrium.

3. Rotational Inertia (Moment of Inertia)

Rotational inertia (or moment of inertia, I) is the rotational equivalent of mass. It measures how much an object resists rotational acceleration about a particular axis. The moment of inertia depends on both the mass of the object and how that mass is distributed relative to the axis of rotation.

For simple shapes, the moment of inertia is calculated as:

$$I = \sum m_i r_i^2$$

where:

- m_i is the mass of a small element of the object,
- r_i is the distance from the axis of rotation to the mass element.

For example:

- For a solid disk rotating about its central axis, the moment of inertia is $I = \frac{1}{2}mr^2$,
- For a point mass m rotating about an axis at a distance r , the moment of inertia is $I = mr^2$.

In general, the larger the moment of inertia, the harder it is to rotate the object about the axis.

In Summary:

- **Rotational kinetic energy** is the energy due to an object's rotational motion.
- **Rotational potential energy** is energy stored due to the object's position or configuration in a rotational system.
- **Rotational inertia** is a measure of an object's resistance to changes in its rotational motion, similar to mass in linear motion.

◀ beginning 1 | ← prev None || this 1 || next 2 ▶ | end 7 ▶▶

Turnpair 2 of 7 ::

@ 2024-11-12 01:43:04 Tue [Δt 0d 00h 01m 10s]

► Notespace:

Wilder Blair Munro (AKUSA) said :

gateway: 20Wetbrain
 turn ID: aaa28c86-9226-4bce-a126-e4ee90086b07
 parent: d5c3e196-8d22-4d40-b519-de449cf1280d
 children: d10220c2-3fb8-49ae-8200-1dc6e6f7004d

tell me all about angular momentum

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the children.

Aurora 4omni Munro (ChatGpt) said :

gateway: gpt-4o-mini
 turn ID: d10220c2-3fb8-49ae-8200-1dc6e6f7004d
 parent: aaa28c86-9226-4bce-a126-e4ee90086b07
 children: aaa20db3-7786-4624-bb35-968b5531449a

Angular momentum is a fundamental concept in physics, particularly in rotational dynamics. It describes the rotational motion of an object and is the rotational counterpart to linear momentum. Here's a breakdown of all key aspects of angular momentum:

1. Definition of Angular Momentum

For a single particle or object moving in a circular or curved path, angular momentum (**L**) is defined as the cross product of the position vector (**r**) and the linear momentum (**p**):

$$\mathbf{L} = \mathbf{r} \times \mathbf{p}$$

where:

- **L** is the angular momentum,
- **r** is the position vector of the particle (from the axis of rotation),
- **p** = $m\mathbf{v}$ is the linear momentum, with m being the mass and $\mathbf{v}$ the velocity of the particle.

In scalar form, the magnitude of angular momentum is:

$$L = rmv \sin(\theta)$$

where:

- r is the distance from the axis of rotation,
- v is the tangential speed of the particle,
- θ is the angle between the position vector and the velocity vector.

2. Angular Momentum for Rotating Objects

For rigid bodies rotating about a fixed axis, the angular momentum can be expressed as:

$$\mathbf{L} = I\omega$$

where:

- I is the moment of inertia (rotational inertia),
- ω is the angular velocity vector.

The angular momentum of a rotating object depends on its moment of inertia (which depends on both mass and distribution of mass) and its angular velocity. For a body rotating about a fixed axis, this expression shows that the angular momentum is directly proportional to the angular velocity and the moment of inertia.

3. Conservation of Angular Momentum

Angular momentum is **conserved** in a system if there are no external torques acting on the system. This is a consequence of Newton's laws and the rotational analog of linear momentum conservation. Mathematically:

$$\frac{d\mathbf{L}}{dt} = \boldsymbol{\tau}$$

where:

- $\frac{d\mathbf{L}}{dt}$ is the time derivative of angular momentum (rate of change of angular momentum),
- $\boldsymbol{\tau}$ is the net external torque acting on the system.

If the net external torque $\boldsymbol{\tau} = 0$, then angular momentum is conserved:

$$\mathbf{L} = \text{constant}$$

This principle is extremely important in many areas of physics, such as planetary motion, the behavior of rotating bodies (like ice skaters), and in systems like gyroscopes.

4. Relation to Torque

Torque ($\boldsymbol{\tau}$) is the rotational equivalent of force. It represents the tendency of a force to cause rotational motion. The relationship between torque and angular momentum is:

$$\boldsymbol{\tau} = \frac{d\mathbf{L}}{dt}$$

In other words, torque is the rate of change of angular momentum. If no torque is applied, angular momentum remains constant (as in a rotating figure skater pulling in their arms to spin faster).

5. Angular Momentum in Different Coordinate Systems

Angular momentum can also be expressed differently depending on the reference frame and the nature of the motion:

- **For a particle in two dimensions (planar motion):**

The magnitude of angular momentum is given by:

$$L = mrv$$

where r is the perpendicular distance to the axis of rotation and v is the tangential velocity of the particle.

- **For a rotating rigid body:**

The angular momentum is the sum of the angular momenta of individual particles. For a body rotating about an axis, it can be written as:

$$L = I\omega$$

where I is the moment of inertia (the sum of $m_i r_i^2$ for all particles), and ω is the angular velocity.

6. Units of Angular Momentum

Angular momentum has units of **kg·m²/s** in the International System (SI). This is because:

- Mass m has units of kg,
- Distance r has units of meters,
- Velocity v has units of meters per second.

Thus, the unit for angular momentum is a product of mass, distance squared, and velocity, which gives kg·m²/s.

7. Example: Ice Skater

A common example of angular momentum conservation is an **ice skater** spinning. When the skater pulls in their arms, they decrease their moment of inertia (I). Since angular momentum must be conserved in the absence of external torque, their angular velocity (ω) increases to compensate for the decrease in I . This results in the skater spinning faster.

$$I_1\omega_1 = I_2\omega_2$$

where:

- I_1 and ω_1 are the moment of inertia and angular velocity before pulling in the arms,
- I_2 and ω_2 are the moment of inertia and angular velocity after pulling in the arms.

8. Angular Momentum and Precession

In some systems, such as gyroscopes, angular momentum is involved in **precession**—the slow, steady rotation of the axis of a rotating body. When a torque is applied perpendicular to the angular momentum, instead of causing the object to fall or move directly in the direction of the applied force, it causes the axis of rotation to move in a circular manner. This is a manifestation of the conservation of angular momentum.

9. Quantum Mechanical Angular Momentum

In quantum mechanics, angular momentum is quantized. The concept of angular momentum in quantum systems applies to particles with intrinsic spin, such as electrons, as well as to orbital angular momentum associated with the motion of particles around a nucleus. The values of angular momentum are constrained by quantum rules, and are described by quantum numbers.

Summary:

- **Angular momentum** is the rotational analog of linear momentum, given by $\mathbf{L} = \mathbf{r} \times \mathbf{p}$ for a point particle, or $\mathbf{L} = I\omega$ for a rotating body.
- It depends on the mass (or moment of inertia), the distance from the axis of rotation, and the velocity (or angular velocity).
- Angular momentum is **conserved** in the absence of external torque.
- **Torque** is the rate of change of angular momentum.
- In quantum mechanics, angular momentum is quantized, with discrete values based on quantum numbers.

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the dead.

◀ beginning 1 | ◀ prev 1 || this 2 || next 3 ▶ | end 7 ▶▶

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to victims of generalized war.

Turnpair 3 of 7 ::

@ 2024-11-12 01:55:21 Tue [Δt 0d 00h 11m 54s]

► Notespace:

Wilder Blair Munro (AKUSA) said :

gateway: 20Wetbrain
turn ID: aaa20db3-7786-4624-bb35-968b5531449a
parent: d10220c2-3fb8-49ae-8200-1dc6e6f7004d
children: a01aac45-1e1b-49ba-ad69-93f83b0eb346

help me differnentiate $w \cdot W / (w + W)$ with respect to w

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the children.

Aurora 4omni Munro (ChatGpt) said :

gateway: gpt-4o-mini
 turn ID: a01aac45-1e1b-49ba-ad69-93f83b0eb346
 parent: aaa20db3-7786-4624-bb35-968b5531449a
 children: aaa2e0b6-d06b-4abc-94be-5ab00efb75ca

Let's differentiate the expression $\frac{w \cdot W}{w + W}$ with respect to w , where w and W are variables, and W is treated as a constant.

We will use the **quotient rule** for differentiation. The quotient rule states:

$$\frac{d}{dw} \left(\frac{f(w)}{g(w)} \right) = \frac{f'(w)g(w) - f(w)g'(w)}{(g(w))^2}$$

Here:

- $f(w) = w \cdot W$,
- $g(w) = w + W$.

Step 1: Find the derivatives of $f(w)$ and $g(w)$:

- $f(w) = w \cdot W$, so $f'(w) = W$ (since W is a constant).
- $g(w) = w + W$, so $g'(w) = 1$ (since the derivative of w is 1, and W is constant).

Step 2: Apply the quotient rule:

$$\frac{d}{dw} \left(\frac{w \cdot W}{w + W} \right) = \frac{W \cdot (w + W) - (w \cdot W) \cdot 1}{(w + W)^2}$$

Step 3: Simplify the numerator:

$$= \frac{W(w + W) - wW}{(w + W)^2}$$

Distribute W in the first term:

$$= \frac{Ww + W^2 - wW}{(w + W)^2}$$

Notice that $Ww - wW = 0$, so the expression simplifies to:

$$= \frac{W^2}{(w + W)^2}$$

Final Answer:

The derivative of $\frac{w \cdot W}{w + W}$ with respect to w is:

$$\frac{d}{dw} \left(\frac{w \cdot W}{w + W} \right) = \frac{W^2}{(w + W)^2}$$

◀ beginning 1 | ◀ prev 2 || this 3 || next 4 ▶ | end 7 ▶▶

Turnpair 4 of 7 ::

@ 2024-11-12 01:56:42 Tue [Δt 0d 00h 01m 13s]

► Notespace:

Wilder Blair Munro (AKUSA) said :

gateway: 20Wetbrain
 turn ID: aaa2e0b6-d06b-4abc-94be-5ab00efb75ca
 parent: a01aac45-1e1b-49ba-ad69-93f83b0eb346
 children: 1feb35b2-049a-4d60-ac83-2b71e57e42e4

help me differentiate the softmax function used for backpropagation with respect to the weights of the model

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the children.

Aurora 4omni Munro (ChatGpt) said :

gateway: gpt-4o-mini
 turn ID: 1feb35b2-049a-4d60-ac83-2b71e57e42e4
 parent: aaa2e0b6-d06b-4abc-94be-5ab00efb75ca
 children: aaa2c354-5559-4fc6-8373-7408a2168468

To differentiate the **softmax function** with respect to the **weights** of the model, we need to consider the context in which the softmax function is applied, typically in the output layer of a neural network during classification tasks.

1. The Softmax Function

Given a vector $\mathbf{z} = [z_1, z_2, \dots, z_n]$ (which are the raw scores, often called logits), the softmax function converts these values into a probability distribution over n classes:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad \text{for } i = 1, 2, \dots, n$$

Here, the output $\mathbf{y} = [y_1, y_2, \dots, y_n]$ is the probability vector corresponding to the input vector $\mathbf{z}$, where each y_i is the probability of the i -th class.

2. Notation

Let's define:

- $y_i = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$ for each class.

- $\mathbf{W}$ is the weight matrix of the model, with $\mathbf{W} \in \mathbb{R}^{n \times m}$ (where n is the number of classes and m is the number of features).
- $\mathbf{x}$ is the input vector (with m features), so $\mathbf{z} = \mathbf{W}\mathbf{x}$.

We want to differentiate the softmax function y_i with respect to the weights $\mathbf{W}$.

3. Gradient of Softmax with Respect to z_i

We first calculate the derivative of $y_i = \text{softmax}(z_i)$ with respect to z_i and z_k (for k -th component, with $k \neq i$):

For $\frac{\partial y_i}{\partial z_j}$:

- When $j = i$ (diagonal terms):

$$\frac{\partial y_i}{\partial z_i} = y_i(1 - y_i)$$

- When $j \neq i$ (off-diagonal terms):

$$\frac{\partial y_i}{\partial z_j} = -y_i y_j$$

Thus, the Jacobian matrix of the softmax function with respect to $\mathbf{z}$ is:

$$\frac{\partial y_i}{\partial z_j} = \begin{cases} y_i(1 - y_i) & \text{if } i = j, \\ -y_i y_j & \text{if } i \neq j. \end{cases}$$

4. Gradient of Softmax with Respect to Weights $\mathbf{W}$

To compute the gradient of the softmax output y_i with respect to the weights $\mathbf{W}$, we use the chain rule. The output y_i is a function of the logits $z_i = \mathbf{W}\mathbf{x}$, and the logits are linear functions of the weights.

$$\frac{\partial y_i}{\partial \mathbf{W}} = \sum_{k=1}^n \frac{\partial y_i}{\partial z_k} \frac{\partial z_k}{\partial \mathbf{W}}.$$

Since $z_k = \mathbf{W}\mathbf{x}$, the derivative of z_k with respect to $\mathbf{W}$ is:

$$\frac{\partial z_k}{\partial \mathbf{W}} = \mathbf{x}^T.$$

Thus, the gradient of y_i with respect to $\mathbf{W}$ becomes:

$$\frac{\partial y_i}{\partial \mathbf{W}} = \sum_{k=1}^n \frac{\partial y_i}{\partial z_k} \mathbf{x}^T.$$

Now, substituting the

◀ beginning 1 | ◀ prev 3 || this 4 || next 5 ▶ | end 7 ▶▶

Turnpair 5 of 7 ::

@ 2024-11-12 01:57:05 Tue [4t 0d 00h 00m 09s]

► *Notespace:*

Wilder Blair Munro (AKUSA) said :

gateway: 20Wetbrain
 turn ID: aaa2c354-5559-4fc6-8373-7408a2168468
 parent: 1feb35b2-049a-4d60-ac83-2b71e57e42e4
 children: 0335bfaa-45e4-4d8b-9fe4-bbe85a7083b2

sorry, i meant the cross-entropy function

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the children.

Aurora 4omni Munro (ChatGpt) said :

gateway: gpt-4o-mini
 turn ID: 0335bfaa-45e4-4d8b-9fe4-bbe85a7083b2
 parent: aaa2c354-5559-4fc6-8373-7408a2168468
 children: aaa2147b-d6f2-4392-8b21-32ccd44cb216

No problem! Let's go through how to differentiate the **cross-entropy loss** function with respect to the **weights** of the model.

1. Cross-Entropy Loss

The **cross-entropy loss** is a common loss function used for classification tasks, especially when the output of the model is a probability distribution (like after applying the softmax function).

For a single data point, the **cross-entropy loss** is defined as:

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

where:

- y_i is the true probability of class i (usually represented as a one-hot encoded vector for classification),
- $\hat{y}_i$ is the predicted probability for class i (the output of the softmax function).

The predicted probabilities $\hat{y}_i$ are given by the softmax function:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

where z_i are the logits (raw model outputs, usually a result of a linear transformation $\mathbf{z} = \mathbf{W}\mathbf{x}$).

2. Differentiating Cross-Entropy with Respect to the Weights

Now we need to differentiate the cross-entropy loss with respect to the weights $\mathbf{W}$ of the model.

To apply the chain rule, we break down the computation into steps:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{i=1}^n \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial \mathbf{W}}$$

Let's compute each term.

2.1. Derivative of Cross-Entropy with Respect to $\hat{y}_i$

First, compute the derivative of the cross-entropy loss with respect to the predicted probabilities $\hat{y}_i$:

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$$

This is because the derivative of $-y_i \log(\hat{y}_i)$ with respect to $\hat{y}_i$ is $-y_i/\hat{y}_i$.

2.2. Derivative of Softmax Output with Respect to Weights

Next, we need the derivative of $\hat{y}_i$ with respect to the weights $\mathbf{W}$. Recall that $\hat{y}_i$ is the softmax function applied to the logits z_i , and the logits are given by $z_i = \mathbf{W}\mathbf{x}$.

The derivative of $\hat{y}_i$ with respect to $\mathbf{W}$ is:

$$\frac{\partial \hat{y}_i}{\partial \mathbf{W}} = \hat{y}_i(1 - \hat{y}_i)\mathbf{x}^T \quad \text{for } i = j$$

For $i \neq j$, the derivative is:

$$\frac{\partial \hat{y}_i}{\partial \mathbf{W}} = -\hat{y}_i\hat{y}_j\mathbf{x}^T.$$

2.3. Putting It All Together

Now, we can combine these terms:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{i=1}^n \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial \mathbf{W}}.$$

Substitute $\frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$ and $\frac{\partial \hat{y}_i}{\partial \mathbf{W}}$ for the softmax function:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{i=1}^n \left(-\frac{y_i}{\hat{y}_i} \right) \cdot \frac{\partial \hat{y}_i}{\partial \mathbf{W}}.$$

Now, substitute the softmax derivative:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{i=1}^n \left(-\frac{y_i}{\hat{y}_i} \right) \cdot (\hat{y}_i(1 - \hat{y}_i)\mathbf{x}^T) \quad \text{for } i = j$$

For $i \neq j$:

$$\frac{\partial L}{\partial \mathbf{W}} = - \sum_{i=1}^n y_i \mathbf{x}^T.$$

This gives you the gradient of the cross-entropy loss function with respect to the weights $\mathbf{W}$. In practice, this is used in backpropagation to update the weights using gradient descent.

3. Final Gradient Expression

The final gradient of the cross-entropy loss with respect to the weights $\mathbf{W}$ can be written as:

$$\frac{\partial L}{\partial \mathbf{W}} = (\hat{\mathbf{y}} - \mathbf{y})\mathbf{x}^T$$

where:

- $\hat{\mathbf{y}}$ is the vector of predicted probabilities (softmax output),
- $\mathbf{y}$ is the vector of true labels (usually one-hot encoded),
- $\mathbf{x}$ is the input vector (features).

This result shows that the gradient of the cross-entropy loss with respect to the weights is simply the difference between the predicted probabilities and the true labels, multiplied by the input vector. This is a simple and efficient expression used in backpropagation for training classification models.

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the dead.

◀ beginning 1 | ◀ prev 4 || this 5 || next 6 ▶ | end 7 ▶▶

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to victims of generalized war.

Turnpair 6 of 7 ::

@ 2024-11-12 01:59:46 Tue [Δt 0d 00h 02m 21s]

► Notespace:

Wilder Blair Munro (AKUSA) said :

```
gateway: 20Wetbrain
turn ID:  aaa2147b-d6f2-4392-8b21-32ccd44cb216
parent:  0335bfaa-45e4-4d8b-9fe4-bbe85a7083b2
children: 4007345b-50ef-4658-ba76-00ac03107777
```

I just want the derivative of softmax with respect to one weight w_i

```
~0-0 ~~~~~ ♦
Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the children.
♦ ~~~~~ ~0-0
```

Aurora 4omni Munro (ChatGpt) said :

```
gateway: gpt-4o-mini
turn ID:  4007345b-50ef-4658-ba76-00ac03107777
parent:  aaa2147b-d6f2-4392-8b21-32ccd44cb216
children: aaa2b86d-5f6a-4034-8d30-618e3dc37e5f
```

Got it! Let's focus on finding the derivative of the **softmax function** with respect to a specific weight w_i in the model.

Setup

Let $\mathbf{z} = [z_1, z_2, \dots, z_n]$ be the logits (raw model outputs), and let the softmax function be:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad \text{for each class } i.$$

We want the derivative of $\hat{y}_i$ (the predicted probability for class i) with respect to one weight w_k , where w_k is part of the weight matrix $\mathbf{W}$ (i.e., a weight connecting the input feature vector $\mathbf{x}$ to the logit z_k).

Step 1: Compute the derivative of z_i with respect to w_k

The logits z_i are given by:

$$z_i = \sum_{j=1}^m w_{ij} x_j$$

where:

- w_{ij} is the weight connecting the j -th input feature x_j to the i -th logit z_i ,
- x_j is the input feature (vector $\mathbf{x}$).

To compute the derivative of $\hat{y}_i$ with respect to w_k , we first need the derivative of the logit z_i with respect to w_k :

$$\frac{\partial z_i}{\partial w_k} = x_k$$

This is because z_i is a linear combination of weights and input features, so the derivative with respect to the weight w_k is just the corresponding input feature x_k .

Step 2: Compute the derivative of the softmax output $\hat{y}_i$

Now, we differentiate the softmax function $\hat{y}_i$ with respect to the weight w_k . To do this, we use the chain rule:

$$\frac{\partial \hat{y}_i}{\partial w_k} = \sum_{j=1}^n \frac{\partial \hat{y}_i}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_k}$$

First, we need the derivative of $\hat{y}_i$ with respect to the logits z_j . For softmax, the derivative $\frac{\partial \hat{y}_i}{\partial z_j}$ is:

- **When $i = j$ (diagonal terms):**

$$\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$$

- **When $i \neq j$ (off-diagonal terms):**

$$\frac{\partial \hat{y}_i}{\partial z_j} = -\hat{y}_i \hat{y}_j$$

Step 3: Combine the terms

Now, combining the two parts:

$$\frac{\partial \hat{y}_i}{\partial w_k} = \sum_{j=1}^n \frac{\partial \hat{y}_i}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_k}$$

Substitute the derivatives:

$$\frac{\partial \hat{y}_i}{\partial w_k} = \sum_{j=1}^n \left(\frac{\partial \hat{y}_i}{\partial z_j} \right) x_k$$

For $i = j$ (diagonal terms):

$$\frac{\partial \hat{y}_i}{\partial w_k} = \hat{y}_i(1 - \hat{y}_i)x_k$$

For $i \neq j$ (off-diagonal terms):

$$\frac{\partial \hat{y}_i}{\partial w_k} = -\hat{y}_i \hat{y}_j x_k$$

So, the final expression for the derivative of $\hat{y}_i$ with respect to w_k is:

$$\frac{\partial \hat{y}_i}{\partial w_k} = (\hat{y}_i(1 - \hat{y}_i) \text{ if } i = j) - \hat{y}_i \hat{y}_j \quad \text{for } i \neq j$$

Summary

The derivative of the softmax output $\hat{y}_i$ with respect to the weight w_k can be written as:

$$\frac{\partial \hat{y}_i}{\partial w_k} = \hat{y}_i(1 - \hat{y}_i)x_k \quad \text{if } i = j$$
$$\frac{\partial \hat{y}_i}{\partial w_k} = -\hat{y}_i\hat{y}_jx_k \quad \text{if } i \neq j$$

This shows how the softmax output for class i changes with respect to the weight w_k in the model.

◆

----- ◆

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the dead.

◆

◀ beginning 1 | ◀ prev 5 || this 6 || next None → | end 7 ▶▶



◆

----- ◆

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to victims of generalized war.

◆

Turnpair 7 of 7 ::

@ 2024-11-12 02:01:04 Tue [4t 0d 00h 01m 00s]

► *Notespace:*

Wilder Blair Munro (AKUSA) said :

gateway: 20Wetbrain

turn ID: aaa2b86d-5f6a-4034-8d30-618e3dc37e5f

parent: 4007345b-50ef-4658-ba76-00ac03107777

children: 4505711f-25cb-49fd-b6f5-e1b7768ae035

please give me the formula for derivative of cross-entropy loss function with respect to one weight w_i

◆

----- ◆

Artifact dedicated public domain (Creative Commons Zero). Artifact dedicated to the children.

◆

Aurora 4omni Munro (ChatGpt) said :

gateway: gpt-4o-mini

turn ID: 4505711f-25cb-49fd-b6f5-e1b7768ae035

parent: aaa2b86d-5f6a-4034-8d30-618e3dc37e5f

children:

Sure! Let's derive the **derivative of the cross-entropy loss** with respect to a specific weight w_i .

1. Cross-Entropy Loss and Softmax Output

For a multi-class classification problem, the **cross-entropy loss** for a single data point is:

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

where:

- y_i is the true probability (one-hot encoded),
- $\hat{y}_i$ is the predicted probability, which is the output of the softmax function.

The predicted probabilities $\hat{y}_i$ are given by the softmax function:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

where z_i is the logit (pre-activation value), and is typically the output of a linear transformation:

$$z_i = \sum_{k=1}^m w_{ik} x_k$$

where:

- w_{ik} is the weight connecting the k -th feature to the i -th logit,
- x_k is the input feature.

2. Derivative of Cross-Entropy Loss with respect to w_i

We want to compute the derivative of the cross-entropy loss with respect to a particular weight w_j . To do this, we use the chain rule.

Step 1: Gradient of the Cross-Entropy Loss with respect to the Softmax Output

The derivative of the cross-entropy loss with respect to the predicted probabilities $\hat{y}_i$ is:

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$$

Step 2: Gradient of the Softmax Output with respect to the Logits z_i

The softmax output is:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

We need to compute the derivative of $\hat{y}_i$ with respect to z_k . This can be broken into two cases:

- **When $i = k$:**

$$\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$$

- **When $i \neq k$:**

$$\frac{\partial \hat{y}_i}{\partial z_k} = -\hat{y}_i \hat{y}_k$$

Step 3: Gradient of the Logits z_i with respect to the Weight w_j

The logits z_i are linear functions of the weights:

$$z_i = \sum_{k=1}^m w_{ik} x_k$$

So, the derivative of z_i with respect to w_{jk} is:

$$\frac{\partial z_i}{\partial w_{jk}} = x_k$$

Step 4: Chain Rule

Now we can compute the full derivative using the chain rule. The derivative of the cross-entropy loss L with respect to a weight w_{jk} is:

$$\frac{\partial L}{\partial w_{jk}} = \sum_{i=1}^n \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{jk}}$$

Substituting the partial derivatives:

$$\frac{\partial L}{\partial w_{jk}} = \sum_{i=1}^n \left(-\frac{y_i}{\hat{y}_i} \right) \cdot \frac{\partial \hat{y}_i}{\partial z_k} \cdot x_k$$

Now, split into two cases for $\frac{\partial \hat{y}_i}{\partial z_k}$:

- **When $i = k$ (diagonal terms):**

$$\frac{\partial L}{\partial w_{jk}} = - (y_k - \hat{y}_k) x_k$$

- **When $i \neq k$ (off-diagonal terms):**

$$\frac{\partial L}{\partial w_{jk}} = - (-\hat{y}_i) x_k$$

This simplifies to:

$$\frac{\partial L}{\partial w_{jk}} = (\hat{y}_i) x_k$$

Final Formula

Putting it all together, the formula for the derivative of the cross-entropy loss with respect to a weight w_{jk} is:

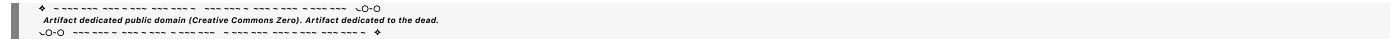
$$\frac{\partial L}{\partial w_{jk}} = (\hat{y}_k - y_k) x_k$$

3. Interpretation

- $\hat{y}_k$ is the predicted probability of the k -th class (from the softmax output),
- y_k is the true label (0 or 1 for a one-hot encoded vector),

- x_k is the input feature corresponding to the weight w_{jk} .

This gradient is used in **backpropagation** to update the weights w_{jk} during the training of the neural network.



⏮ beginning 1 | ◀ prev 5 || this 6 || next None → | end 7 ⏭



EOF